



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки Кафедра
інформаційних систем та технологій

Лабораторна робота №5
із дисципліни «**Технології розроблення програмного забезпечення**»
Патерни проектування.

Виконав:

студент групи ІА–34

Оборський Дмитро Вікторович

Київ 2025

Тема: Патерни проектування.

Мета: Вивчити структуру шаблонів «Adapter», «Builder», «Command», «Chain of responsibility», «Prototype» та навчитися застосовувати їх в реалізації програмної системи.

14. Архіватор (strategy, adapter, factory method, facade, visitor, p2p)

Архіватор повинен являти собою візуальний додаток з можливістю створення і редагування архівів різного типу (.tar.gz, .zip, .rar, .ace) – додавання/ видалення файлів / папок, редагування метаданих (по можливості), перевірка checksum архівів, тестування архівів на наявність пошкоджень, розбиття архівів на частини.

Теоретичні відомості

1.1.1. Шаблон «Adapter»

Призначення патерну: Шаблон "Adapter" (Адаптер) використовується для адаптації інтерфейсу одного об'єкту до іншого [6]. Наприклад, існує декілька бібліотек для роботи з принтерами, проте кожна має різний інтерфейс (хоча однакові можливості і призначення). Має сенс розробити уніфікований інтерфейс (сканування, асинхронне сканування, двостороннє сканування, потокове сканування і тому подібне), і реалізувати відповідні адаптери для приведення бібліотек до уніфікованого інтерфейсу. Це дозволить в програмі звертатися до загального інтерфейсу, а не приводити різні сценарії роботи залежно від способу реалізації бібліотеки. Адаптери також називаються "wrappers" (обгортками).

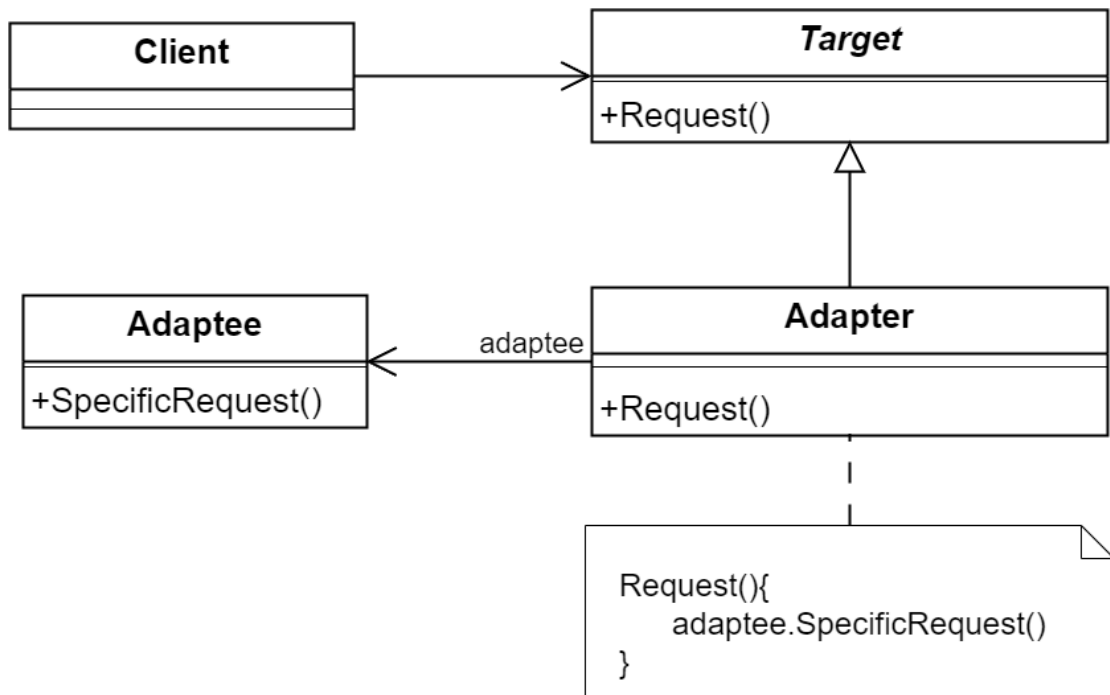


Рисунок 5.1. Структура патерну Адаптер на рівні об'єктів

Проблема: Ви реалізовуєте аудіо-плеєр, який може програвати аудіо різних форматів. Ви почали аналізувати і бачите що для програвання аудіо із різних форматів краще використовувати різні компоненти.

Таким чином ви реалізуєте складну логіку роботи з різними компонентами. У вас у коді з'являється багато логіки з перевіркою, якщо формат один, то викликаємо такий метод у такого компонента, якщо інший, то викликаємо декілька методів у іншого компонента, і т.д. Логіка роботи з компонентами стає досить складною та заплутаною.

Коли потрібно додати підтримку нового аудіо-формату, то потрібно додати роботу з новим компонентом і при цьому внести зміну в уже існуючу логіку, що може привести до помилок там де все коректно працювало.

Рішення: Для вирішення цих проблем можна використати патерн Адаптер. Визначаємо загальний інтерфейс `IPlayer` для програвання музики через компоненти. Далі для кожного компонента робимо свій адаптер. Адаптер має посилання на об'єкт компонента та реалізує інтерфейс викликаючи методи з компонента, який він адаптує. Таким чином, адаптер ніби обгортає специфічний

компонент і далі весь клієнтський код буде працювати з адаптерами через інтерфейс IPlayer. Класи, які будуть працювати з адаптером через цей інтерфейс не будуть знати інтерфейси кожного специфічного компонента.

При такій реалізації, якщо буде потрібно додати підтримку нового формату, то просто створюється новий адаптер, який обгортає новий компонент, але робота з цим адаптером буде виконуватися через існуючий інтерфейс. При цьому існуючий код не буде зазнавати змін, а значить і не "поламаємо" те що вже працює.

Переваги та недоліки:

- + Відокремлює інтерфейс або код перетворення даних від основної бізнес-логіки.
- + Можна добавляти нові адаптери не змінюючи код у класі Client.
- Умовним недоліком можна назвати збільшення кількості класів, але за рахунок використання патерна Адаптер програмний код, як правило, стає легше читати.

1.1.2. Шаблон «Builder»

Призначення патерну: Шаблон «Builder» (Будівельник) використовується для відділення процесу створення об'єкту від його представлення [6]. Це доречно у випадках, коли об'єкт має складний процес створення (наприклад, Web-сторінка як елемент повної відповіді web- сервера) або коли об'єкт повинен мати декілька різних форм створення (наприклад, при конвертації тексту з формату у формат).

Проблема: Візьмемо процес побудови відповіді на запит web-сервера. Побудова складається з наступних частин: додавання стандартних заголовків (дата/час, ім'я сервера, інш.), код статусу (після пошуку відповідної сторінки на сервері), заголовки відповіді (тип вмісту, інш.), утримуване, інше.

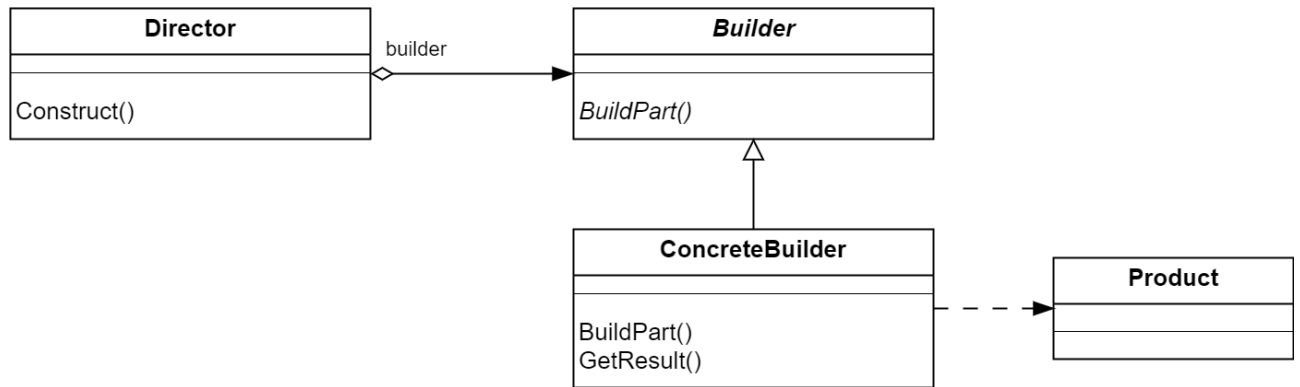


Рисунок 5.2. Структура патерну Builder

Рішення: Кожен з цих етапів може бути абстрагований в окремий метод будівельника. Це дасть наступні вигоди:

- Гнучкіший контроль над процесом створення сторінки;
- Незалежність від внутрішніх змін – наприклад, зміна назви сервера не сильно порушить процес побудови відповіді;

Переваги та недоліки:

+ Дозволяє використовувати один і той самий код для створення різноманітних продуктів.

- Клієнт буде прив'язаний до конкретних класів будівельників, тому що в інтерфейсі будівельника може не бути методу отримання результату.

1.1.3. Шаблон «Command»

Призначення патерну: Шаблон "command" (команда) перетворить звичайний виклик методу в клас [6]. Таким чином дії в системі стають повноправними об'єктами. Це зручно в наступних випадках:

- Коли потрібна розвинена система команд – відомо, що команди будуть добавлятися;
- Коли потрібна гнучка система команд – коли з'являється необхідність додавати командам можливість відміни, логування і інш.;
- Коли потрібна можливість складання ланцюжків команд або виклику команд в певний час.

Об'єкт команда сама по собі не виконує ніяких фактичних дій окрім перенаправлення запиту одержувачеві (тобто команди все ж виконуються одержувачем), однак ці об'єкти можуть зберігати дані для підтримки додаткових функцій відміни, логування і інш. Наприклад, команда вставки символу може запам'ятовувати символ, і при виклику відміни викликати відповідну функцію витирання символу. Можна також визначити параметр «застосовності» команди (наприклад, на картинці писати не можна) – і використати цей атрибут для засвічування піктограми в меню.

Такий підхід до команд дозволяє побудувати дуже гнучку систему команд, що настроюється. У більшості додатків це буде зайвим (використовується спрощений варіант), проте життєво важливий в додатках з великою кількістю команд (редактори).

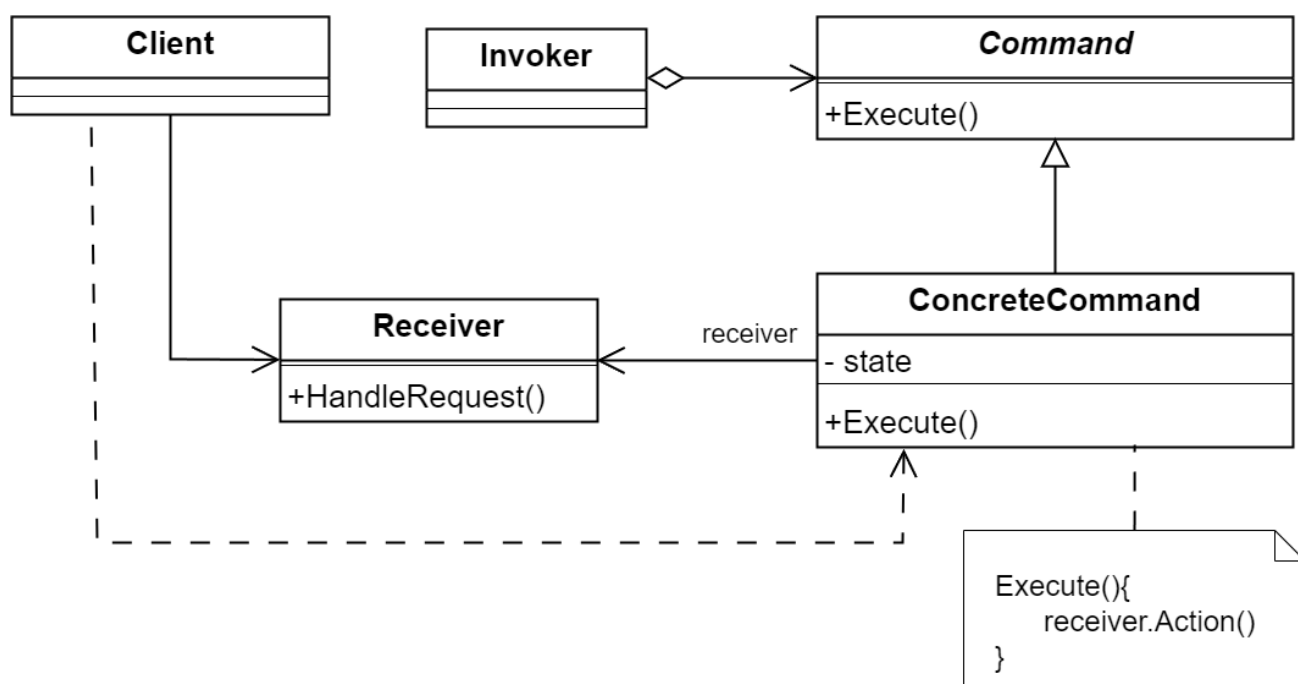


Рисунок 5.3. Структура патерну Команда

Проблема: Ви реалізуєте товстий клієнт який має багатий візуальний інтерфейс: має меню, кнопки і контекстне меню. Кожна дія, яку можна виконати, має три варіанти виконання – через меню, натисканням кнопки та через контекстне меню. Реалізацію кожної дії можна розмістити в обробнику

візуального елементу, але тоді потрібно буде продублювати цей функціонал для меню, кнопки та контекстного меню. Таким чином ми будемо мати дублювання коду і при зміні функціоналу змінювати його в трьох місцях.

Додатковим викликом буде реалізувати автоматизоване тестування такої системи, тому що нам необхідно емулювати натискання кнопок користувачем.

Рішення: Виділення функціоналу який виконується по натисканню на кнопку в окремий клас дозволяє відв'язати візуальну частину від логіки обробки. Таким чином ми будемо мати шар з UI елементами і шар з логікою обробки дій виконаних на UI. Це дозволяє один і той самий об'єкт з шару логіки обробки дій використати для реакцію на натискання кнопки і пункту меню і контекстного меню. Кнопки тепер не знають про конкретні класи реалізації команд, а взаємодіють з об'єктами команд через загальний інтерфейс. Ще одна перевага, що тепер ми можемо достатньо просто реалізувати механізм enable-disable для кнопок та контекстного меню через виклик у об'єкта команди метода IsEnabled() або через прив'язку (binding) на поле IsEnabled у об'єкта команди.

При такій реалізації також набагато простіше організувати тестування застосунку, тому що ми тепер не потрібно емулювати дії користувача, а достатньо протестувати шар логіки обробки використовуючи модульні тести (unit tests).

Переваги та недоліки:

- + Ініціатор виконання команди не знає деталей реалізації виконавця команди.
- + Підтримує операції скасування та повторення команд.
- + Послідовність команд можна логувати і при необхідності виконати цю послідовність ще раз.
- + Простота розширення за рахунок додавання нових команд без необхідності внесення змін в уже існуючий код (принцип відкритості-закритості).

1.1.4. Шаблон «Chain of Responsibility»

Призначення патерну: Шаблон «Chain of responsibility» (Ланцюжок відповідальності) частково можна спостерігати в житті, коли підписання відповідного документу проходить від його складання у одного із співробітників компанії через менеджера і начальника до головного начальника, який ставить свій підпис [5].

Проблема: Ви розробляєте систему зі складними UI формами, які містять багато вкладених один в один візуальних компонентів, по кліку правою кнопкою миші вам потрібно сформувати контекстне меню. В залежності від того на якому компоненті був виконаний клік мишкою, вміст контекстного меню повинен відрізнятися, також в контекстне меню потрібно добавляти пункти, якщо компонент, в який входить поточний, теж має свої пункти. Один із очевидних підходів – ви робити метод, який викликається по кліку мишки. В методі ви робити перевірки і якщо в даний момент конкретний елемент, добавляєте в контекстне меню підходящі до нього пункти, потім перевіряєте інші елементи і перевіряєте чи вони не є такими, що містять в собі вибраний елемент. Якщо так, то добавляєте в контекстне меню ще пункти.

З часом алгоритм формування меню стає більш складним, тому що добавляються нові елементи. Також з часом в алгоритмі залишилися перевірки на компоненти, які з форми були вже видалені.

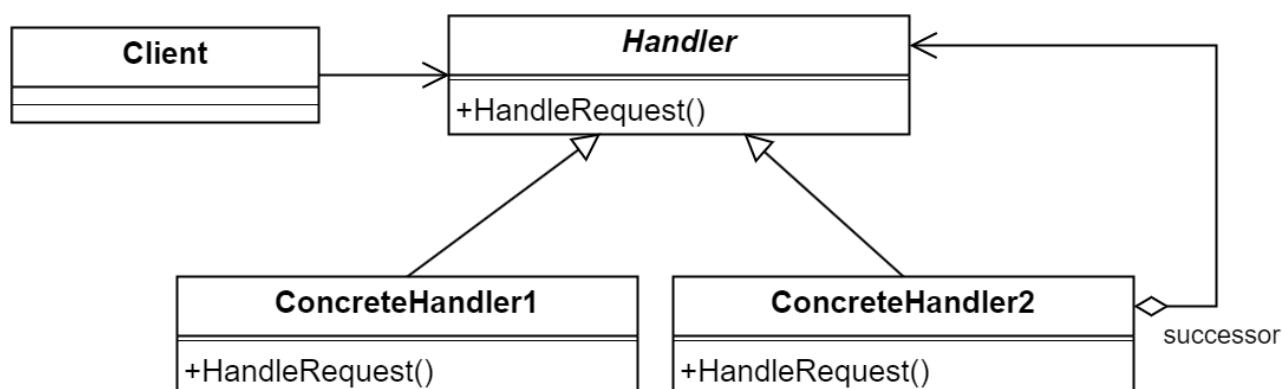


Рисунок 5.4. Структура патерна Ланцюжок відповідальності

Рішення: Для вирішення проблеми зі зростаючою складністю формування контекстного меню підходить патерн «Ланцюжок відповідальності».

Основна ідея в тому, що нам не потрібно мати загальний метод формування контекстного меню. Ми можемо зробити загальний інтерфейс з методом `UpdateContextMenu()` і реалізувати цей метод в усіх візуальних компонентах. Додатково для візуальних компонентів додаємо поле для переходу на "батьківський" елемент, якій в собі містить поточний візуальний компонент, а в реалізації `UpdateContextMenu` в кінці додаємо виклик цього метода у "батьківського" елемента. Якщо елемент не потребує додавання пунктів в контекстне меню, він просто викликає цей метод у наступного елемента в ланцюжку. Тепер обробка правого кліку мишки буде виглядати наступним чином: По кліку правою кнопкою миші викликаємо `UpdateContextMenu` на тексті, він, наприклад, додає пункт меню «Копіювати текст», далі викликає аналогічний метод у компонента «текст-блок» в якому цей текст знаходиться, але текст-блок нічого в контекстне меню не добавляє, а викликає `UpdateContextMenu` в елементі `Grid`, в якому знаходиться текст-блок, а `Grid` при відпрацюванні метода добавляє пункт меню «Контекстна допомога» і так далі.

Коли ми змінюємо структуру форми, то між компонентами змінюються зв'язки в ланцюжку, але самі алгоритми вже переробляти не потрібно. якщо видаляється з форми якийсь компонент, то разом з ним видаляється і логіка пов'язана з цим компонентом. Таким чином складність роботи з контекстним меню залишається контрольованою.

Слід додати що елемент, що обробляє виклик, не обов'язково повинен передавати виклик далі по ланцюжку. Це залежить від того, яку поведінку ви хочете отримати.

Переваги та недоліки:

- + Зменшує залежність між клієнтом та обробниками: клієнт не знає хто обробить запит, а обробники не знають структуру ланцюжка.
- + Реалізовує додаткову гнучкість в обробці запиту: легко додати або вилучити з ланцюжка нові обробники.

- Запит може залишитися ніким не опрацьованим: запит не має вказаного обробника, тому може бути не опрацьованим.

1.1.5. Шаблон «Prototype»

Призначення патерну: Шаблон «Prototype» (Прототип) використовується для створення об'єктів за «шаблоном» (чи «кресленням», «ескізом») шляхом копіювання шаблонного об'єкту, який називається прототипом. Для цього визначається метод «клонувати» в об'єктах цього класу [6].

Цей шаблон зручно використати, коли заздалегідь відомо як виглядатиме кінцевий об'єкт (мінімізується кількість змін до об'єкту шляхом створення шаблону), а також для видалення необхідності створення об'єкту – створення відбувається за рахунок клонування, і самій програмі немає необхідності знати, як створювати об'єкт.

Також, це дозволяє маніпулювати об'єктами під час виконання програми шляхом настроювання відповідних прототипів; значно зменшується ієрархія наслідування (оскільки в іншому випадку це були б не прототипи, а вкладені класи, що наслідують).

Проблема: Ви розробляєте редактор рівнів для 2D гри на основі спрайтів. В панелі інструментів ви маєте багато кнопок для різних елементів, які можна розташовувати на екрані, такі як сходи, стіни, підлога, оздоблення та інші. Ці елементи у вас об'єднані в ієрархію з базовим класом `GameObject`. Під кожен елемент можна зробити свій тип кнопки, але тоді ми отримаємо паралельну ієрархію кнопок і при додаванні нового типу ігрового об'єкту потрібно буде додавати і новий тип кнопки.

Рішення: Використовуючи патерн прототип, додаємо до базового об'єкта `GameObject` метод `Clone()`, а кнопки будуть зберігати посилання на об'єкт базового типу `GameObject`. При натисканні на кнопку об'єкт який потрібно додати на ігровому полі отримуємо не створенням нового, а клонуванням прототипу, який прив'язаний до кнопки. Таким чином, коли ми будемо додавати

нові типи ігрових об'єктів, то логіка роботи з кнопками не буде змінюватися, тому що не має прив'язки до конкретних типів.

Також слід відзначити, що при копіюванні об'єкту, навіть приватні поля будуть скопійовані, тому що реалізація методу копіювання знаходиться в цьому класі, що копіюється і таким чином є доступ для копіювання і до відкритих і до закритих полів.

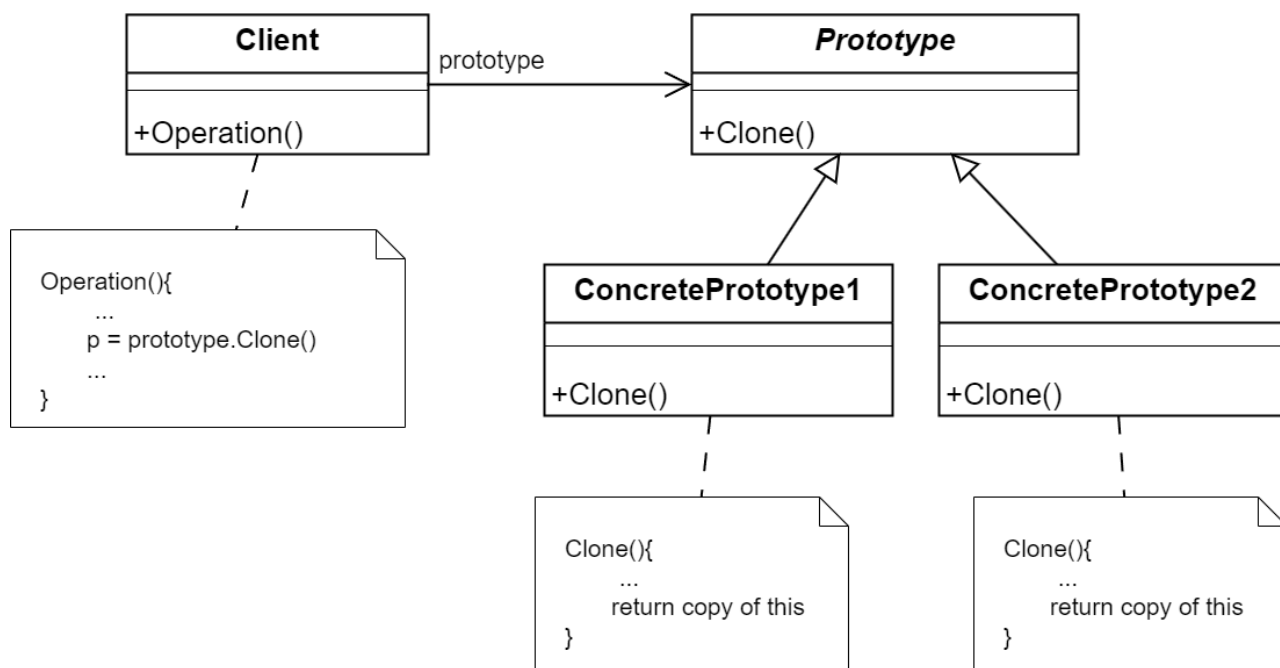


Рисунок 5.5. Структура патерну «Прототип»

Переваги та недоліки:

- + За рахунок клонування складних об'єктів замість їх створення, підвищується продуктивність.
- + Різні варіації об'єктів можна отримувати за рахунок клонування, а не розширення ієрархії класів.
- + Вища гнучкість, тому що клоновані об'єкти можна модифікувати незалежно, не впливаючи на об'єкт з якого була зроблена копія.
- Реалізація глибокого клонування досить проблематична, коли об'єкт що клонується містить складну внутрішню структуру та посилання на інші об'єкти.

- Надмірне використання патерну Прототип може привести до ускладнення коду та проблем із супроводом такого коду.

Хід роботи

1. Створюю ZipAdapter

```
1 package com.beowulf.core.adapter;
2
3 import org.apache.commons.compress.archivers.zip.ZipArchiveEntry;
4 import org.apache.commons.compress.archivers.zip.ZipArchiveOutputStream;
5 import org.apache.commons.compress.archivers.zip.ZipFile;
6 import org.apache.commons.io.IOUtils;
7
8 import java.io.*;
9 import java.nio.file.*;
10 import java.util.Enumeration;
11 import java.util.Objects;
12
13 You, 7 hours ago | 1 author (You)
14 public class ZipAdapter {
15     /**
16      * Compresses sourceDir into a .zip archive.
17      *
18      * @param sourceDir path to file or directory to compress.
19      * @param targetArchive path to resulting .zip file.
20      */
21     public void compress(Path sourceDir, Path targetArchive) throws IOException {
22         Objects.requireNonNull(sourceDir, "sourceDir");
23         Objects.requireNonNull(targetArchive, "targetArchive");
24
25         if (!Files.exists(sourceDir)) {
26             throw new FileNotFoundException("Source directory not found: " + sourceDir);
27         }
28
29         Path parent = sourceDir.getParent();
30         Path base = parent != null ? parent : sourceDir;
31
32         try (OutputStream outputStream = Files.newOutputStream(targetArchive);
33             BufferedOutputStream bufferedOutputStream = new BufferedOutputStream(outputStream);
34             ZipArchiveOutputStream zipArchiveOutputStream = new ZipArchiveOutputStream(bufferedOutputStream)) {
35             zipArchiveOutputStream.setLevel(ZipArchiveOutputStream.STORED);
36
37             Files.walk(sourceDir)
38                 .forEach(path -> {
39                     try {
40                         Path relativePath = base.relativeTo(path);
41                         String entryName = relativePath.toString().replace(File.separatorChar, '/');
42
43                         if (entryName.isEmpty()) {
44                             return;
45                         }
46
47                         if (Files.isDirectory(path)) {
48                             if (!entryName.endsWith("/")) {
49                                 entryName = entryName + "/";
50                             }
51                             You, 7 hours ago • ✨ Set up db + modified compressing
52                             ZipArchiveEntry dirEntry = new ZipArchiveEntry(entryName);
53                             dirEntry.setTime(Files.getLastModifiedTime(path).toMillis());
54                             zipArchiveOutputStream.putArchiveEntry(dirEntry);
55                             zipArchiveOutputStream.closeArchiveEntry();
56                         } else {
57                             try (InputStream fileInputStream = Files.newInputStream(path)) {
58                                 ZipArchiveEntry entry = new ZipArchiveEntry(entryName);
59                                 zipArchiveOutputStream.putArchiveEntry(entry);
60                                 IOUtils.copy(fileInputStream, zipArchiveOutputStream);
61                                 zipArchiveOutputStream.closeArchiveEntry();
62                             }
63                         }
64                     } catch (IOException error) {
```

Рис. 1 ZipAdapter частина 1

```

13 public class ZipAdapter {
21     public void compress(Path sourceDir, Path targetArchive) throws IOException {
49         if (!entryName.endsWith("/")) {
50             entryName = entryName + "/";
51             // You, 7 hours ago + ⚡ Setted up db + modified compressing
52             ZipArchiveEntry dirEntry = new ZipArchiveEntry(entryName);
53             dirEntry.setTime(Files.getLastModifiedTime(path).toMillis());
54             zipArchiveOutputStream.putArchiveEntry(dirEntry);
55             zipArchiveOutputStream.closeArchiveEntry();
56         } else {
57             try (InputStream fileInputStream = Files.newInputStream(path)) {
58                 ZipArchiveEntry entry = new ZipArchiveEntry(entryName);
59                 zipArchiveOutputStream.putArchiveEntry(entry);
60                 IOUtils.copy(fileInputStream, zipArchiveOutputStream);
61                 zipArchiveOutputStream.closeArchiveEntry();
62             }
63         }
64     } catch (IOException error) {
65         throw new UncheckedIOException("ZIP compression failed", error);
66     }
67 });
68
69     zipArchiveOutputStream.finish();
70 }
71 }
72
73 /**
74  * Extracts a .zip archive.
75  *
76  * @param archive path to existing .zip archive.
77  * @param targetDir destination directory.
78  */
79 public void decompress(Path archive, Path targetDir) throws IOException {
80     Objects.requireNonNull(archive);
81     Objects.requireNonNull(targetDir);
82
83     if (!Files.exists(archive)) {
84         throw new FileNotFoundException("Archive file not found: " + archive);
85     }
86
87     Files.createDirectories(targetDir);
88
89     try (ZipFile zipFile = ZipFile.builder().setPath(archive).get()) {
90         Enumeration<ZipArchiveEntry> entries = zipFile.getEntries();
91
92         while (entries.hasMoreElements()) {
93             ZipArchiveEntry entry = entries.nextElement();
94             Path outputPath = targetDir.resolve(entry.getName());
95
96             if (entry.isDirectory()) {
97                 Files.createDirectories(outputPath);
98             } else {
99                 Files.createDirectories(outputPath.getParent());
100                 try (InputStream inputStream = zipFile.getInputStream(entry);
101                     OutputStream outputStream = Files.newOutputStream(outputPath)) {
102                     IOUtils.copy(inputStream, outputStream);
103                 }
104             }
105         }
106     }
107 }
108 }
109 }
110

```

Рис. 2 ZipAdapter часть 2

2. Створюю TarGzAdapter

```
1 package com.beowulf.core.adapter;
2
3 import org.apache.commons.compress.archivers.ArchiveEntry;
4 import org.apache.commons.compress.archivers.tar.TarArchiveEntry;
5 import org.apache.commons.compress.archivers.tar.TarArchiveInputStream;
6 import org.apache.commons.compress.archivers.tar.TarArchiveOutputStream;
7 import org.apache.commons.io.IOUtils;
8
9 import java.io.*;
10 import java.nio.file.*;
11 import java.util.Objects;
12 import java.util.zip.GZIPInputStream;
13 import java.util.zip.GZIPOutputStream;
14
15 You, 7 hours ago | 1 author (You)
16 public class TarGzAdapter {
17     /**
18      * Compresses sourceDir into a .tar.gz archive.
19      *
20      * @param sourceDir path to file or directory to compress.
21      * @param targetArchive path to resulting .tar.gz file.
22      */
23     public void compress(Path sourceDir, Path targetArchive) throws IOException {
24         Objects.requireNonNull(sourceDir, "sourceDir");
25         Objects.requireNonNull(targetArchive, "targetArchive");
26
27         if (!Files.exists(sourceDir))
28             throw new FileNotFoundException("Source directory not found: " + sourceDir);
29
30         Path parent = sourceDir.getParent();
31         Path base = parent != null ? parent : sourceDir;
32
33         try (OutputStream filOutputStream = Files.newOutputStream(targetArchive);
34             BufferedOutputStream bufferedOutputStream = new BufferedOutputStream(filOutputStream);
35             GZIPOutputStream gzipOutputStream = new GZIPOutputStream(bufferedOutputStream);
36             TarArchiveOutputStream tarArchiveOutputStream = new TarArchiveOutputStream(gzipOutputStream)) {
37
38             tarArchiveOutputStream.setLongFileMode(TarArchiveOutputStream.LONGFILE_GNU);
39
40             Files.walk(sourceDir)
41                 .forEach(path -> {
42                     try {
43                         Path relativePath = base.relativize(path);
44                         String entryName = relativePath.toString().replace(File.separatorChar, '/');
45
46                         if (entryName.isEmpty()) {
47                             return;
48                         }
49
50                         if (Files.isDirectory(path)) {
51                             if (!entryName.endsWith("/")) {
52                                 entryName = entryName + "/";
53                             }
54
55                             TarArchiveEntry dirEntry = new TarArchiveEntry(entryName);
56                             dirEntry.setModTime(Files.getLastModifiedTime(path).toMillis());
57                             tarArchiveOutputStream.putArchiveEntry(dirEntry);
58                             tarArchiveOutputStream.closeArchiveEntry();
59                         } else {
60                             TarArchiveEntry fileEntry = new TarArchiveEntry(path.toFile(), entryName);
61                             tarArchiveOutputStream.putArchiveEntry(fileEntry);
62                             try (InputStream in = Files.newInputStream(path)) {
63                                 IOUtils.copy(in, tarArchiveOutputStream);
64                             }
65                             tarArchiveOutputStream.closeArchiveEntry();
66                         }
67                     } catch (IOException e) {
68                         // Handle exception
69                     }
70                 });
71         }
72     }
73 }
```

Рис.3 TarGzAdapter частина 1

```

15 public class TarGzAdapter {
23     public void compress(Path sourceDir, Path targetArchive) throws IOException {
49
50         if (Files.isDirectory(path)) {
51             if (!entryName.endsWith("/")) {
52                 entryName = entryName + "/";
53             }
54
55             TarArchiveEntry dirEntry = new TarArchiveEntry(entryName);
56             dirEntry.setModTime(Files.getLastModifiedTime(path).toMillis());
57             tarArchiveOutputStream.putArchiveEntry(dirEntry);
58             tarArchiveOutputStream.closeArchiveEntry();
59         } else {
60             TarArchiveEntry fileEntry = new TarArchiveEntry(path.toFile(), entryName);
61             tarArchiveOutputStream.putArchiveEntry(fileEntry);
62             try (InputStream in = Files.newInputStream(path)) {
63                 IOUtils.copy(in, tarArchiveOutputStream);
64             }
65             tarArchiveOutputStream.closeArchiveEntry();
66         }
67     } catch (IOException e) {
68         throw new UncheckedIOException("TAR.GZ compression failed", e);
69     }
70 }
71
72     tarArchiveOutputStream.finish();
73 }
74
75
76 /**
77  * Extracts a tar.gz archive.
78  *
79  * @param archive path to existing .tar.gz archive.
80  * @param targetDir destination directory.
81  */
82 public void decompress(Path archive, Path targetDir) throws IOException {
83     Objects.requireNonNull(archive);
84     Objects.requireNonNull(targetDir);
85
86     if (!Files.exists(archive))
87         throw new FileNotFoundException("Archive file not found: " + archive);
88
89     Files.createDirectories(targetDir);
90
91     try (InputStream fileInputStream = Files.newInputStream(archive);
92         BufferedInputStream bufferedInputStream = new BufferedInputStream(fileInputStream);
93         GZIPInputStream gzipInputStream = new GZIPInputStream(bufferedInputStream);
94         TarArchiveInputStream tarInputStream = new TarArchiveInputStream(gzipInputStream)) {
95
96         ArchiveEntry entry;
97         while ((entry = tarInputStream.getNextEntry()) != null) {
98             Path outputPath = targetDir.resolve(entry.getName());
99
100             if (entry.isDirectory()) {
101                 Files.createDirectories(outputPath);
102             } else {
103                 Files.createDirectories(outputPath.getParent());
104                 try (OutputStream outputStream = Files.newOutputStream(outputPath)) {
105                     IOUtils.copy(tarInputStream, outputStream);
106                 }
107             }
108         }
109     }
110 }
111 }
112

```

Рис. 4 TarGzAdapter часть 2

3. Створюю RarAdapter

```
You, last week | 1 author (You)
1 package com.beowulf.core.adapter;
2
3 import java.io.*;
4 import java.nio.file.*;
5 import java.util.*;
6
You, last week | 1 author (You)
7 public class RarAdapter {
8
9     /**
10      * Compresses sourceDir into a .rar archive using external "rar" tool.
11      * Requires user to have rar installed on their OS.
12      *
13      * @param sourceDir path to file or directory to compress.
14      * @param targetArchive path to resulting .rar file.
15      */
16     public void compress(Path sourceDir, Path targetArchive) throws IOException {
17         Objects.requireNonNull(sourceDir, "sourceDir");
18         Objects.requireNonNull(targetArchive, "targetArchive");
19
20         if (!Files.exists(sourceDir)) {
21             throw new FileNotFoundException("Source directory not found: " + sourceDir);
22         }
23
24         String rar = findRarBinary()
25             .orElseThrow(() -> new IOException("""
26                 'rar' binary is not installed or not in PATH.
27
28                 Please install it manually:
29                 • Arch Linux: yay -S rar
30                 • Debian/Ubuntu: Download 'rarlinux' package from https://rarlab.com and install manually
31                 • macOS: brew install --cask rar
32                 • Windows: Install WinRAR (includes rar.exe)
33                 """));
34
35         List<String> cmd = new ArrayList<>();
36         cmd.add(rar);
37         cmd.add("a");
38         cmd.add("-ep1");
39         cmd.add(targetArchive.toAbsolutePath().toString());
40         cmd.add(sourceDir.toAbsolutePath().toString());
41
42         runProcess(cmd);
43     }
44
45     /**
46      * Extracts a .rar archive using "rar x".
47      * Requires user to have rar installed on their OS.
48      *
49      * @param archive path to existing .rar archive.
50      * @param targetDir destination directory.
51      */
52     public void decompress(Path archive, Path targetDir) throws IOException {
53         Objects.requireNonNull(archive, "archive");
54         Objects.requireNonNull(targetDir, "targetDir");
55
56         if (!Files.exists(archive)) {
57             throw new FileNotFoundException("Archive file not found: " + archive);
58         }
59
60         Files.createDirectories(targetDir);
61
62         String rar = findRarBinary()
63             .orElseThrow(() -> new IOException("""
64                 'rar' binary is not installed or not in PATH.
65                 """));
66     }
67 }
```

Рис. 5 RarAdapter частина 1


```

63         .orElseThrow(() -> new IOException("
64         'rar' binary is not installed or not in PATH.
65
66         Please install it manually:
67         • Arch Linux:      yay -S rar
68         • Debian/Ubuntu:  Download 'rarlinux' package from https://rarlab.com and install manually
69         • macOS:          brew install --cask rar
70         • Windows:        Install WinRAR (includes rar.exe)
71         ""));
72
73     List<String> cmd = new ArrayList<>();
74     cmd.add(rar);
75     cmd.add("x");
76     cmd.add("-o+");
77     cmd.add(archive.toAbsolutePath().toString());
78     cmd.add(targetDir.toAbsolutePath().toString() + File.separator);
79
80     runProcess(cmd);
81 }
82
83 private Optional<String> findRarBinary() {
84     if (isWindows()) {
85         if (isOnPath(binary: "rar.exe"))
86             return Optional.of("rar.exe");
87         if (isOnPath(binary: "rar"))
88             return Optional.of("rar");
89     } else {
90         if (isOnPath(binary: "rar"))
91             return Optional.of("rar");
92     }
93     return Optional.empty();
94 }
95
96 private boolean isOnPath(String binary) {
97     try {
98         Process process = new ProcessBuilder(binary).start();
99         process.destroy();
100        return true;
101    } catch (IOException error) {
102        return false;
103    }
104 }
105
106 private boolean isWindows() {
107     String os = System.getProperty("os.name").toLowerCase(Locale.ROOT);
108     return os.contains("win");
109 }
110
111 private void runProcess(List<String> command) throws IOException {
112     ProcessBuilder processBuilder = new ProcessBuilder(command);
113     processBuilder.redirectErrorStream(true);
114
115     Process process = processBuilder.start();
116
117     StringBuilder output = new StringBuilder();
118     try (BufferedReader reader = new BufferedReader(new InputStreamReader(process.getInputStream()))) {
119         String line;
120         while ((line = reader.readLine()) != null) {
121             output.append(line).append(System.lineSeparator());
122         }
123     }
124
125     try {
126         int exit = process.waitFor();
127         if (exit != 0) {
128             throw new IOException("RAR failed (exit " + exit + "):\n" + output);
129         }
130     } catch (InterruptedException error) {
131         process.destroyForcibly();
132         Thread.currentThread().interrupt();
133         throw new IOException("RAR interrupted", error);
134     }
135 }
136

```

Рис. 6 RarAdapter часть 2

4. Створюю AceAdapter. Так як .ace формат є пропрієтарним і належить GmbH, то єдине що ми можемо запропонувати користувачу це декомпресію використовуючи їхню офіційну бібліотеку

```
1 package com.beowulf.core.adapter;
2
3 import java.io.*;
4 import java.nio.file.*;
5 import java.util.*;
6
7 You, last week | 1 author (You)
8 public class AceAdapter {
9     /**
10      * Extracts .ace archive using external "unace" tool.
11      * Requires user to have unace installed on their OS.
12      *
13      * @param archive path to existing .ace archive.
14      * @param targetDir destination directory.
15      */
16     public void decompress(Path archive, Path targetDir) throws IOException {
17         Objects.requireNonNull(archive, "archive");
18         Objects.requireNonNull(targetDir, "targetDir");
19
20         if (!Files.exists(archive)) {
21             throw new FileNotFoundException("Archive file not found: " + archive);
22         }
23
24         Files.createDirectories(targetDir);
25
26         String unace = findUnaceBinary()
27             .orElseThrow(() -> new IOException("""
28                 'unace' binary is not installed or not in PATH.
29
30                 Please install it manually:
31                 • Arch Linux:    sudo pacman -S unace
32                 • Debian/Ubuntu: sudo apt install unace-nonfree
33                 • macOS:       brew install unace
34                 • Windows:     Install WinACE (includes unace.exe)
35                 """));
36
37         List<String> cmd = new ArrayList<>();
38         cmd.add(unace);
39         cmd.add("x");
40         cmd.add(archive.toAbsolutePath().toString());
41         cmd.add(targetDir.toAbsolutePath().toString() + File.separator);
42
43         runProcess(cmd);
44     }
45
46     private Optional<String> findUnaceBinary() {
47         if (isWindows()) {
48             if (isOnPath(binary: "unace.exe"))
49                 return Optional.of("unace.exe");
50             if (isOnPath(binary: "unace"))
51                 return Optional.of("unace");
52         } else {
53             if (isOnPath(binary: "unace"))
54                 return Optional.of("unace");
55         }
56         return Optional.empty();
57     }
58
59     private boolean isOnPath(String binary) {
60         try {
61             Process process = new ProcessBuilder(binary).start();
62             process.destroy();
63             return true;
64         } catch (IOException error) {
65             return false;
66         }
67     }
68 }
```

Рис. 7 AceAdapter частина 1

```

7 public class AceAdapter {
16     public void decompress(Path archive, Path targetDir) throws IOException {
37         List<String> cmd = new ArrayList<>(),
38         cmd.add(unace);
39         cmd.add("x");
40         cmd.add(archive.toAbsolutePath().toString());
41         cmd.add(targetDir.toAbsolutePath().toString() + File.separator);
42
43         runProcess(cmd);
44     }
45
46     private Optional<String> findUnaceBinary() {
47         if (isWindows()) {
48             if (isOnPath(binary: "unace.exe"))
49                 return Optional.of("unace.exe");
50             if (isOnPath(binary: "unace"))
51                 return Optional.of("unace");
52         } else {
53             if (isOnPath(binary: "unace"))
54                 return Optional.of("unace");
55         }
56         return Optional.empty();
57     }
58
59     private boolean isOnPath(String binary) {
60         try {
61             Process process = new ProcessBuilder(binary).start();
62             process.destroy();
63             return true;
64         } catch (IOException error) {
65             return false;
66         }
67     }
68
69     private boolean isWindows() {
70         String os = System.getProperty("os.name").toLowerCase(Locale.ROOT);
71         return os.contains("win");
72     }
73
74     private void runProcess(List<String> command) throws IOException {
75         ProcessBuilder processBuilder = new ProcessBuilder(command);
76         processBuilder.redirectErrorStream(true);
77
78         Process process = processBuilder.start();
79
80         StringBuilder output = new StringBuilder();
81         try (BufferedReader reader = new BufferedReader(new InputStreamReader(process.getInputStream()))) {
82             String line;
83             while ((line = reader.readLine()) != null) {
84                 output.append(line).append(System.lineSeparator());
85             }
86         }
87
88         try {
89             int exit = process.waitFor();
90             if (exit != 0) {
91                 throw new IOException("ACE failed (exit " + exit + "):\n" + output);
92             }
93         } catch (InterruptedException error) {
94             process.destroyForcibly();
95             Thread.currentThread().interrupt();
96             throw new IOException("ACE interrupted", error);
97         }
98     }
99 }
100

```

Рис. 8 AceAdapter часть 2

У ході цієї лабораторної роботи я закріпив теоретичні знання про шаблони «Adapter», «Builder», «Command», «Chain of responsibility», «Prototype» та на практиці зосередився на реалізації саме патерну «Adapter» у проєкті архіватора Beowulf. На його основі були створені адаптери ZipAdapter, TarGzAdapter, RarAdapter та AceAdapter, які уніфікують роботу з різними архівними форматами, навіть якщо під капотом використовуються зовнішні утиліти чи бібліотеки. Загалом робота продемонструвала, що патерни проєктування це зручний інструмент для побудови гнучкого й розширюваного ПЗ.

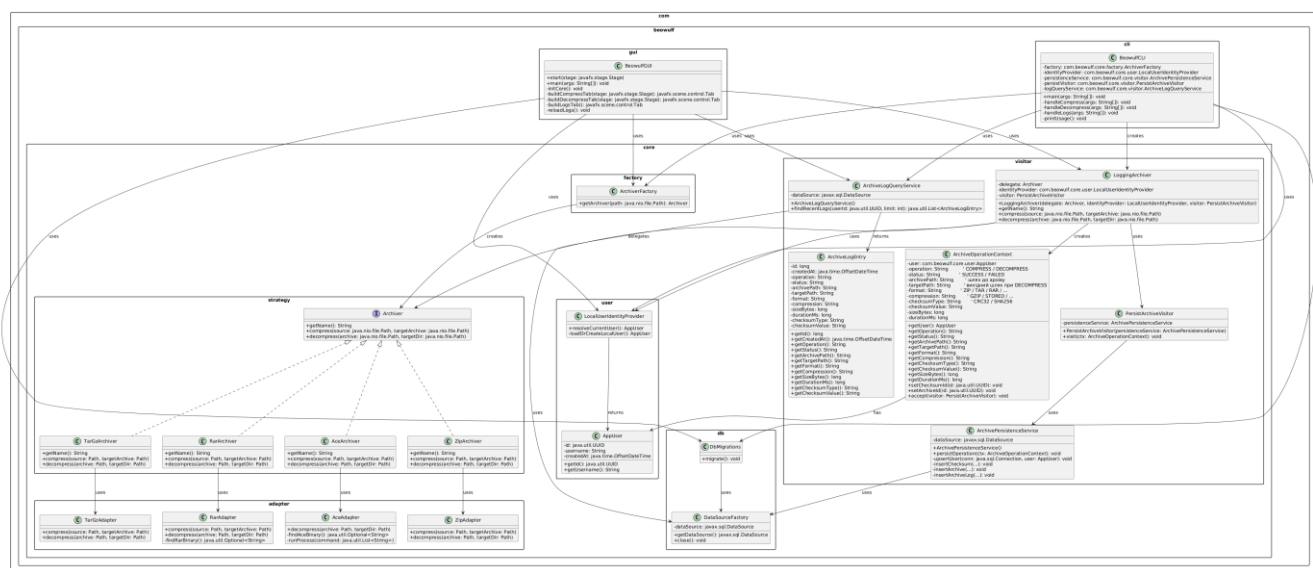


Рис. 6 Діаграма класів

Висновок

У ході цієї лабораторної роботи я закріпив теоретичні знання про шаблони «Adapter», «Builder», «Command», «Chain of responsibility», «Prototype» та на практиці зосередився на реалізації саме патерну «Adapter» у проєкті архіватора Beowulf. На його основі були створені адаптери ZipAdapter, TarGzAdapter, RarAdapter та AceAdapter, які уніфікують роботу з різними архівними форматами, навіть якщо під капотом використовуються зовнішні утиліти чи бібліотеки. Загалом робота продемонструвала, що патерни проєктування це зручний інструмент для побудови гнучкого й розширюваного ПЗ.

Контрольні запитання

1. Яке призначення шаблону «Адаптер»?

(Щоб перетворити інтерфейс одного класу в інший, сумісний із кодом клієнта, тобто «обгорнути» сторонній/несумісний API так, щоб система працювала через єдиний узгоджений інтерфейс.)

2. Нарисуйте структуру шаблону «Адаптер».

(Є інтерфейс Target, клієнт працює з ним; є клас Adaptee з «незручним» інтерфейсом; клас Adapter реалізує Target і всередині делегує виклики методам Adaptee.)

3. Які класи входять в шаблон «Адаптер», та яка між ними взаємодія?

(Класи: Client, інтерфейс Target, Adaptee і Adapter; клієнт викликає методи Target, насправді працюючи з Adapter, а той у своїх методах викликає потрібні методи Adaptee.)

4. Яка різниця між реалізацією «Адаптера» на рівні об'єктів та на рівні класів?

(Об'єктний адаптер зберігає посилання на Adaptee і використовує композицію; класовий адаптер наслідує Adaptee і Target (або імплементує/наслідує їх одночасно), що жорсткіше зв'язує типи.)

5. Яке призначення шаблону «Будівельник»?

(Відокремити процес побудови складного об'єкта від його представлення, дозволяючи конструювати різні варіанти «продукту» одним і тим самим алгоритмом кроків.)

6. Нарисуйте структуру шаблону «Будівельник».

(Є інтерфейс Builder з методами типу buildPartA(), buildPartB(), конкретні ConcreteBuilderX, що збирають різні варіанти продукту, клас Director, який викликає кроки будівництва в певному порядку, та клас Product, що є результатом.)

7. Які класи входять в шаблон «Будівельник», та яка між ними взаємодія?

(Класи: Director, інтерфейс Builder, ConcreteBuilder, Product; Director керує послідовністю викликів методів Builder, конкретний будівельник створює й накопичує частини Product, а потім повертає готовий об'єкт.)

8. У яких випадках варто застосовувати шаблон «Будівельник»?

(Коли створення об'єкта має багато кроків/параметрів, потрібні різні варіації результату й треба уникнути «конструкторів-монстрів» із купою аргументів.)

9. Яке призначення шаблону «Команда»?

(Інкапсулювати запит у вигляді об'єкта, щоб можна було передавати, ставити в чергу, логувати, відмінати й комбінувати дії незалежно від відправника та отримувача.)

10. Нарисуйте структуру шаблону «Команда».

(Інтерфейс Command з методом execute(), конкретні команди ConcreteCommandX, які тримають посилання на Receiver і в execute() викликають його методи, Invoker, що зберігає команду і викликає execute(), та Receiver, який виконує реальну роботу.)

11. Які класи входять в шаблон «Команда», та яка між ними взаємодія?

(Класи: `Invoker`, `Command/ConcreteCommand`, `Receiver`; клієнт налаштовує `Invoker` певною командою, `Invoker` викликає `command.execute()`, а команда делегує дію на `Receiver`.)

12. Розкажіть як працює шаблон «Команда».

(Кожна дія оформлюється як об'єкт-команда з методом `execute()` і, за потреби, `undo()`. Інтерфейс/кнопка/меню лише викликають команду, а логіка дії й стан для відміни зберігаються всередині об'єкта-команди, що дозволяє їх складати, логувати й повторювати.)

13. Яке призначення шаблону «Прототип»?

(Створювати нові об'єкти шляхом клонування вже наявного зразка-прототипу, не прив'язуючись до конкретних конструкторів і мінімізуючи витрати на складне створення.)

14. Нарисуйте структуру шаблону «Прототип».

(Інтерфейс/абстрактний клас `Prototype` з методом `clone()`, конкретні `ConcretePrototypeX`, що реалізують `clone()`; клієнт зберігає десь посилання на прототип і при потребі викликає `prototype.clone()` замість `new`.)

15. Які класи входять в шаблон «Прототип», та яка між ними взаємодія?

(Класи: `Prototype` і його нащадки `ConcretePrototype`, плюс `Client`; клієнт налаштований на роботу з об'єктами-прототипами і створює нові екземпляри, викликаючи на них `clone()`.)

16. Які можна привести приклади використання шаблону «Ланцюжок відповідальності»?

(Наприклад: обробка HTTP-запиту через кілька фільтрів/ middleware; проброс події в UI через дерево компонентів; послідовна перевірка запиту різними валідаторами або ролями доступу; погодження документа, коли його по черзі розглядають різні посадові особи.)